

I started my ML in the second year of college. Due to a lack of resources, I went through courses on YouTube and built small projects like a movie recommendation system, IPL score prediction, etc. Then, when I started my journey as a Quantum ML intern, I started to know that there are other different methods like SVM (which can be implemented using Quantum ML). So then I started to search more in this ML field and got to know about more models. While reading different research papers, I found it interesting: why can't we predict the most probable places where we can find archaeological sites? As a resident of India, we have a great history which has been recorded, like the Ramayana, Mahabharata, etc. So why can't I pick the places and the suitable conditions where we can find an old civilization? Then I started my project EpicGeo.

I built a pipeline that extracts geographic information from Sanskrit epics—NER to find place mentions, geocoding to locate them, some uncertainty modelling (since "seven days east" obviously isn't a coordinate), and rasterisation to get everything into a format a model can use. Then I put together nine feature datasets: geology, land use, river proximity, reconstructed ancient routes, and a co-mention graph of which places show up together across texts. A Random Forest does the actual classification, trained against known Indian sites.

Honestly, picking the model is where I noticed my biggest gap. I went from thinking regression, to decision trees, to random forests—mostly by trying things and seeing what stuck. Looking back, if my foundations had been solid from the start, I probably could have reasoned my way straight to a random forest, or maybe something better, instead of stumbling into it. It's the same story in other parts of the project. My uncertainty modelling was more improvised than principled, and spatial data has issues like autocorrelation and leakage between nearby points that I've only patched up informally. I want to get to a point where my choices come from actually understanding the theory, not just habit.

That's what I'm hoping to get out of MLSS. I've already shown I can take a hard problem and get it working on my own—what I'm missing is the rigorous foundation that turns a working pipeline into real research, and people who'll push back on my reasoning instead of just nodding along.

I don't think the best fit for a program like this is whoever already knows the most. I think it's someone with a real problem in hand, who's already gotten it partly working, and who has specific gaps they actually want to close. That feels like me.